

Chosen Scope and Motivation

The chosen scope for my project is: Academic Assistant for IIT Bombay. The system built is designed to answer the queries related to UG and PG academic regulations, course policies, semester timelines, academic procedures, IDDDP guidelines and other institute-level academic information.

Why did I choose this scope?

Because it seemed to be a more helpful and useful solution to create out of the others. Students are usually unaware about stuff like these and popular LLMs can't solve insti based problems. Moreover, I was also very easily able to find documents related to this scope I used the following:

UG rulebook

Academic Calendar

IDDDP guidelines

Academic Procedures and Punishments

Rules for Academic Awards

To solve this problem, I implemented a RAG pipeline. Instead of depending only on internal knowledge of the language model, relevant information is retrieved from verified institute documents at query time and is provided as additional context before generating answer.

The goal of IITB Insti-Assist is to create a grounded assistant that answers only using retrieved information. If the needed information isn't available in provided knowledge base, the system avoids hallucination and just says that it doesn't know.

In the project I have also implemented additional features such as source attribution, confidence scoring, and user PDF upload support.

Data Sources Used

The data sources have been mentioned above.

The documents were collected as PDF and processed in document ingestion pipeline. The raw text was extracted from these PDFs and converted into smaller chunks before indexing.

Along with fixed IITB knowledge base, assistant also supports live PDF upload. A user can upload any new document after which the system extracts content, creates temporary embeddings, builds searchable FAISS index, and performs question answering over the uploaded document. This allows the same architecture to work not only for IITB documents but also for any custom domain knowledge source.

Chunking Strategy

Directly storing complete documents for retrieval is inefficient because of largeness of these documents like UG rulebook. Retrieving an entire document gives excessive irrelevant information to LLM and reduce answer accuracy. To solve this chunking is done like this:

Paragraphs are accumulated together until chunk size reached a maximum threshold of 300 characters. After this a 150 character overlap from the previous chunk was retained while creating the next chunk.

Smaller chunks allow FAISS to retrieve only most relevant sections instead of entire documents. The language model receives focused context, reducing chances of hallucination. Similar topics are grouped together and chunk overlap helps prevent loss of information when important content appears near the chunk boundaries.

After chunking, each text segment becomes a dense vector representation using a transformer based MiniLM sentence embedding model. These capture semantic meaning instead of only matching keywords.

The generated embeddings are stored in a FAISS vector index. When a user asks a question the query will also be converted into an embedding and similarity search is performed to retrieve the closest matching chunks. So finally, retrieved context is then passed to the LLM, to generate answers grounded in the original documents.

Known Limitations and Future Improvements

Although the project demonstrates a working RAG based assistant successfully, there are some limitations still. The accuracy of the system depends on the quality of extracted text from PDFs. Documents with complex format, tables, images or scanned pages may not be parsed correctly, which affects retrieval quality.

The current retrieval system uses dense vector similarity search. While this works well for semantic queries but some cases involving exact keywords, course codes or specific identifiers may benefit from combining vector search with the known traditional keyword search.

The system also uses a locally stored FAISS index, which works well for a small collection of documents but it may require a dedicated vector database for scaling to more number of documents. So, future improvements include:

Adding OCR support for scanned documents and image based PDFs

Implementing hybrid retrieval using both keyword search and vector similarity

Adding reranking models to improve the ordering of retrieved chunks

Deploying the assistant as a publicly accessible web application (right now its on local host)